

PRÁTICAS DE PROGRAMAÇÃO · AULA 03

Função lambda & Tratamento de exceções

Capítulos 5 e 6 · Práticas de Programação · Editora Senac SP

Prof. Rafael Pereira

Onde estamos na jornada

O que já construímos e para onde isso vai

Aula 01

Funções · escopo · listas ·
matrizes



Aula 02

Fatiamento · tuplas ·
dicionários



Aula 03

Lambda · exceções



Oficina

PTI: Controle de Estoque

Lembrete: a armadilha das aspas

Copiou do PDF? “ ” ‘ ’ não são aspas do Python.

SyntaxError: invalid character '""' (U+201C)

Ctrl+H → substituir “ e ” por " (e ‘ ’ por ').

Sua bagagem para hoje

- def / return / parâmetros (aula 01)
- lista, .append(), for (aula 01)
- dicionário: produto["preco"] (aula 02)

Tudo isso aparece de novo hoje — dentro de lambda e dentro de try.

O PTI usa os 4 blocos da jornada. Depois da aula de hoje, vocês têm 100% da teoria necessária para entregar.

Caça ao erro: o livro tem 2 bugs hoje

Exemplo 1 do cap. 5 — digite exatamente como está na p. 2 e rode

```
def soma(x, y):  
    return x + y  
print(soma(3, 4))    # Saída: 7
```

IndentationError: expected an indented block after function definition on line 1

como está impresso

```
def soma(x, y):  
    return x + y      # 4 espaços!  
print(soma(3, 4))  
7
```

corrigido

Por que isso é conteúdo, não só um erro de impressão

IndentationError é a única exceção da aula que acontece ANTES do programa rodar — é erro de sintaxe, não de execução. Guarde essa distinção: daqui a 20 minutos ela é o primeiro conceito do cap. 6. O 2º bug está no Exemplo 2 do cap. 6 (p. 3). Quem achar primeiro, avisa.

Bug nº 3, de conceito: a p. 8 fala em "funções map() e lambda()". lambda não é função — é palavra-chave. lambda() dá SyntaxError.

Função lambda: a função de uma linha

lambda parâmetros: expressão

```
# jeito tradicional
def soma(x, y):
    return x + y

# jeito lambda (mesma coisa)
soma = lambda x, y: x + y

print(soma(3, 4))
7
print(type(soma))
<class 'function'>
```

lambda **x**, **y**: **x** + **y**

palavra-chave

parâmetros (sem parênteses)

expressão = valor devolvido (sem return)

3 coisas que lambda NÃO faz

1. Não tem return — a expressão já é o retorno.
2. Não tem 2ª linha nem 2ª instrução (if/for/print ; return → SyntaxError).
3. Não tem nome próprio: <function <lambda>> — o nome vem da variável.

Quando usar lambda (e quando não)

Não é desempenho — o livro é claro: def e lambda rodam igual. É legibilidade.

Use lambda quando

- A função cabe em uma expressão curta.
- Ela é descartável: vai ser usada uma vez, agora.
- Ela é argumento de outra função:
sort(key=...), map(...), filter(...)

Use def quando

- Precisa de mais de uma linha ou de if/for.
- Vai reutilizar em vários lugares.
- Quer nome e docstring (o PTI é assim!).
- Alguém vai ter que depurar aquilo.

```
# a mesma lambda, usada como argumento (o uso real dela)
print((lambda x: x * 3)(5))          # chamada direta: 15
print(list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4, 5, 6])))
[2, 4, 6]
```

Regra prática para o PTI: lambda só dentro do sum(map(...)) do total. Todo o resto com def.

sort() altera · sorted() devolve novo

Exemplo 2 do cap. 5 — e a armadilha que o livro não mostra

```
numeros = [5, 2, 9, 1, 5, 6]

numeros.sort()          # mexe na própria lista
print(numeros)
[1, 2, 5, 5, 6, 9]

nova = sorted(numeros, reverse=True)  # lista NOVA
print(nova)
[9, 6, 5, 5, 2, 1]
print(numeros)          # a original não mudou
[1, 2, 5, 5, 6, 9]
```

Armadilha: sort() devolve None

```
numeros = numeros.sort()
print(numeros)
None
```

A lista sumiu. `sort()` ordena e devolve nada.

Lembra da aula 01?

É o mesmo caso de `.append()`: método que altera a lista não devolve a lista. Mesma pegadinha, outro nome.

key=lambda: você escolhe o critério

É aqui que lambda brilha — como argumento de outra função

```
peessoas = [("Alice", 25), ("Bob", 30), ("Carol", 20)]
peessoas.sort(key=lambda x: x[1])    # x = cada tupla; x[1] = idade
print(peessoas)
[('Carol', 20), ('Alice', 25), ('Bob', 30)]
```

```
# PONTE PARA O PTI: lista de dicionários, ordenada pelo preço
estoque = [
    {"codigo": 1, "nome": "Mouse",      "preco": 45.0, "quantidade": 10},
    {"codigo": 2, "nome": "Teclado",    "preco": 120.0, "quantidade": 3},
    {"codigo": 3, "nome": "Cabo HDMI",  "preco": 25.5, "quantidade": 7},
]
por_preco = sorted(estoque, key=lambda p: p["preco"])
print([p["nome"] for p in por_preco])
['Cabo HDMI', 'Mouse', 'Teclado']
```

Como ler `key=lambda x: x[1]`: "para cada item x, use `x[1]` como critério de comparação". O `sort` chama a lambda uma vez por item.

map() aplica · list() materializa

Exemplos 3, 4, 5 e 6 — e o que o livro não mostra: o iterador esgota

```
numeros = [1, 2, 3, 4, 5]
print(map(lambda x: x * 2, numeros))
<map object at 0x7f8b...>      # não é lista ainda!
print(list(map(lambda x: x * 2, numeros)))
[2, 4, 6, 8, 10]

print(list(map(str.upper, ["alice", "bob"])))
['ALICE', 'BOB']              # sem lambda
print(list(range(5)))
[0, 1, 2, 3, 4]
```

Iterador esgota

```
v = map(lambda x: x**2,
[1,2,3,4])
print(list(v))
[1, 4, 9, 16]
print(list(v))
[]
```

Só pode consumir 1 vez.

Ponte para o PTI

```
sum(map(lambda p:
p["quantidade"], estoque))
→ 20
```

É a opção 2 do menu. Uma linha.

map(função, iterável): aplica a função item a item e devolve um iterador preguiçoso. list() força o cálculo.

Celsius → Fahrenheit em 3 versões

```
celsius = [0, 10, 20, 30, 40, 100]

# v1 - imperativo (Ex. 7)
fahrenheit = []
for temp in celsius:
    fahrenheit.append((temp * 9/5) + 32)

# v2 - função def (Ex. 8)
def celsius_para_fahrenheit(c):
    return (c * 9/5) + 32
fahrenheit = []
for temp in celsius:
    fahrenheit.append(celsius_para_fahrenheit(temp))

# v3 - map + lambda (Ex. 9)
fahrenheit = list(map(lambda c: (c * 9/5) + 32, celsius))
print(fahrenheit)
```

Saída (as 3 iguais)

```
[32.0, 50.0, 68.0,
 86.0, 104.0, 212.0]
```

Por que 32.0? Porque 9/5 é divisão real → float.

Critério de conclusão

- ✓ As 3 versões imprimem a mesma lista.
- ✓ Você sabe dizer qual linha da v3 substitui o for da v1.

Extra: reescreva v3 com def em vez de lambda: map(celsius_para_fahrenheit, celsius).

Exceção não é erro de sintaxe

Cap. 6 — o programa estava certo, o mundo é que não colaborou

Erro de sintaxe

Acontece ANTES de rodar. O Python nem começa.
Ex.: `IndentationError`, `SyntaxError`, aspas “ ”.
Quem corrige: você, no editor.

```
x = 1 / 0
print('nunca chega aqui')
```

ZeroDivisionError: division by zero

Exceção

Acontece DURANTE a execução, com código correto.
Ex.: usuário digitou "abc", arquivo sumiu, dividiu por 0.
Quem trata: o programa, com `try/except`.

```
try:
    x = 1 / 0
except ZeroDivisionError:
    print("Erro: divisão por zero não permitida!")
print('o programa continua')
```

Erro: divisão por zero não permitida!
o programa continua

`try` = "vigie este bloco". `except TipoDaExceção` = "se ESSA exceção acontecer, faça isto". O programa segue vivo.

Vários except · else · finally

Exemplo 2 (com o bug do livro corrigido) e Exemplo 3

```
try:
    num = int(input("Digite um número: "))
    resultado = 10 / num
except ValueError:
    print("Erro: digite um inteiro válido.")
except ZeroDivisionError:
    print("Erro: divisão por zero.")
else:
    print(f"Resultado: {resultado}") # ← bug do livro

5    → Resultado: 2.0
0    → Erro: divisão por zero.
abc  → Erro: digite um inteiro válido.
2.5  → Erro: digite um inteiro válido.
-3   → Resultado: -3.3333333333333335 (passou!)
```

try roda o bloco vigiado

except só se deu erro — e só o except do tipo certo. Ordem importa: o 1º que casar vence.

else só se NÃO deu erro nenhum

finally SEMPRE, com ou sem erro (fechar arquivo, avisar fim)

```
except FileNotFoundError: # Ex. 3
    print("Arquivo não encontrado.")
finally:
    print("Encerrando o processo.")
```

Bug nº 2 do livro (p. 3): o print do else está sem indentação → IndentationError. Repare também que -3 passa: inteiro válido ≠ valor aceitável.

Quadro 1, versão de bolso

O livro lista 20 exceções. Estas 6 são as que vocês vão ver de verdade este semestre

Exceção	Quando acontece	Como provocar (teste você)
<code>ValueError</code>	valor de tipo certo, conteúdo errado	<code>int("abc") · float("79,90")</code>
<code>ZeroDivisionError</code>	divisão por zero	<code>10 / 0</code>
<code>KeyError</code>	chave não existe no dicionário	<code>produto["preco"]</code> (sem essa chave)
<code>IndexError</code>	posição fora da lista	<code>lista[100]</code>
<code>TypeError</code>	tipo errado na operação	<code>"5" + 3</code>
<code>FileNotFoundError</code>	arquivo não existe	<code>open("x.txt")</code>

Correção ao Quadro 1 do livro

`IOError` aparece como exceção separada, mas no Python 3 é apenas um apelido: `IOError` is `OSError` → True. `FileNotFoundError` é filha de `OSError`.

Exception é a mãe de todas. Por isso `except Exception` pega tudo — e por isso o livro (p. 6) manda evitar.

Leitor que não morre: ler_inteiro()

```
def ler_inteiro(mensagem):  
    while True:                # insiste até conseguir  
        try:  
            return int(input(mensagem))    # deu certo? sai  
        except ValueError:  
            print("    [!] Digite um número inteiro.")
```

```
idade = ler_inteiro("Sua idade: ")  
print("ok, idade =", idade)
```

```
Sua idade: abc  
    [!] Digite um número inteiro.  
Sua idade: 2.5  
    [!] Digite um número inteiro.  
Sua idade:          (Enter vazio)  
    [!] Digite um número inteiro.  
Sua idade: 7  
ok, idade = 7
```

Como funciona

while True gira para sempre.
O return dentro do try é a única saída.
Se int() falhar, cai no except, imprime e o while gira de novo.

Critério de conclusão

✓ abc, 2.5 e Enter vazio não derrubam o programa.
✓ 7 é aceito.

Desafio: e -3? Passa. Anote — é a prática 3.

raise: você decide o que é erro

Exemplo 4 — a regra de negócio vira exceção

```
def verificar_idade(idade):  
    if idade < 18:  
        raise ValueError("Idade deve ser maior ou  
igual a 18.")  
    return "Acesso permitido."  
  
try:  
    print(verificar_idade(16))  
except ValueError as e:  
    print(f"Erro: {e}")  
Erro: Idade deve ser maior ou igual a 18.  
  
print(verificar_idade(20))  
Acesso permitido.
```

raise = lançar

O Python não sabe que 16 é inválido. Você sabe. raise avisa: "isto é um erro, e a mensagem é esta".

A execução da função PARA ali — o return nunca roda.

except ... as e

e é a exceção capturada, guardada numa variável. print(e) mostra só a mensagem — a mesma que você escreveu no raise.

Sem o try: traceback e programa morto.

Regra: quem valida levanta (raise); quem chama captura (except). A função não imprime erro — ela avisa.

Boas práticas e logging

O que o livro pede na p. 6 — e o Exemplo 5

Evite: except Exception

```
try:
    print(nome_inexistente)
except Exception:
    print("deu algum erro")
```

NameError escondido. Você nunca vai achar.

Prefira

- Capturar o tipo específico (ValueError).
- Mensagem que diz o que aconteceu e o que fazer:
"o preço não pode ser negativo" ✓
"erro" ✗

```
import logging
logging.basicConfig(level=logging.ERROR)
try:
    x = 1 / 0
except ZeroDivisionError as e:
    logging.error(f"Erro detectado: {e}")
ERROR:root:Erro detectado: division by zero
```

Por que logging e não print?

Níveis: DEBUG < INFO < WARNING < ERROR < CRITICAL. Com level=ERROR, warning() some. Sai pela saída de erro (stderr): no VS Code pode aparecer em outra cor.

No PTI, print() nas mensagens ao usuário é correto — é interface. logging é para o desenvolvedor.

validar_preco(): o coração do PTI

```
def validar_preco(texto):
    preco = float(texto.replace(",", "."))    # aceita 79,90
    if preco < 0:
        raise ValueError("o preço não pode ser negativo")
    return preco

def ler_preco(mensagem):
    while True:
        try:
            return validar_preco(input(mensagem))
        except ValueError as e:
            print(f"    [!] Preço inválido: {e}")

print("preço lido:", ler_preco("Preço: "))
Preço: -10
    [!] Preço inválido: o preço não pode ser negativo
Preço: xyz
    [!] Preço inválido: could not convert string to float: 'xyz'
Preço: 79,90
preço lido: 79.9
```

2 ValueError, 1 except

`float("xyz")` levanta o do Python.

`raise` levanta o seu.

O mesmo `except` pega os dois — e `{e}` mostra a mensagem certa.

Critério de conclusão

✓ -10 recusado com SUA mensagem.

✓ xyz recusado com a do Python.

✓ 79,90 vira 79.9.

Guarde este código: é a leitura de preço do PTI.

Hoje você aprendeu a

Escrever curto. Falhar com elegância.

Lambda para o curto. try/except/raise para a elegância. O PTI precisa dos dois — e a oficina é a parte 2 deste material.

Referências: GUILHON et al. Jornada Python. Brasport, 2022 · LAMBERT. Fundamentos de Python. Cengage, 2022 · Práticas de Programação, cap. 5 e 6, Editora Senac SP.

PARTE 2 · OFICINA

Agora, o projeto.

Sistema de Controle de Estoque — 7 funções, 8 requisitos, 50 minutos, construído junto.

Antes de começar: todos com o `aula03.py` salvo (a prática 3 vai ser reaproveitada) e o enunciado do PTI aberto ao lado. Arquivo novo: `controle_estoque.py`.

Sistema de Controle de Estoque

Produção Textual Individual · Prof. autor: Alexandre dos Santos Mignon · Senac EAD

Regra da oficina: construímos juntos, uma função por vez, testando cada uma antes da próxima. Ao final, cada um sai com o SEU arquivo rodando — e sabendo explicar cada linha.

O enunciado, linha por linha

Cada frase do professor autor vira um item que o avaliador vai procurar

#	O que o enunciado exige	Como o avaliador confere
R1	Python, modo console	roda no terminal, sem interface gráfica
R2	usa funções, listas e dicionários (obrigatório)	há def, [] e {} no código
R3	estoque = lista de dicionários com código, nome, preço, quantidade	estoque = [] e cada produto é um dict com as 4 chaves
R4	menu com opção 1: Cadastrar Produto	existe uma função só para isso
R5	não permitir código duplicado	cadastrar o mesmo código 2x é recusado
R6	preço e quantidade não negativos	digitar -1 é recusado, sem travar
R7	menu com opção 2: Calcular Total de Produtos	soma das quantidades, em uma função
R8	dados só durante a execução	sem arquivo, sem banco: lista em memória

Entrega: PDF com nome completo + código-fonte, no item Produção Textual Individual. Prazo: calendário de atividades.

Você já sabe tudo que precisa

Cada requisito $\leftrightarrow$ o capítulo em que aprendemos

Aula 01 · Cap. 1–2

```
def, parâmetros, return,  
escopo  
lista, .append(), for  
→ R2, R3, R4, R7, R8
```

Aula 02 · Cap. 3–4

```
dicionário, produto["chave"]  
.strip(), .replace()  
→ R3, R5
```

Aula 03 · Cap. 5–6

```
try / except / raise  
while True + return  
sum(map(lambda))  
→ R6, R7
```

O que NÃO vamos usar (de propósito)

Classes, arquivos, banco de dados, bibliotecas externas, variáveis globais. O enunciado não pede, o livro ainda não ensinou, e cada um deles é uma chance de errar. Escopo (cap. 1): estoque entra nas funções como PARÂMETRO.

Regra de ouro do cap. 1 aplicada: nenhuma função acessa estoque "de fora". Ela recebe e devolve.

Arquitetura: 7 funções e um main()

O mapa do arquivo que vamos construir de baixo para cima

`main()`

`cria estoque = []`
`while True: menu`

`exibir_menu()`

só print

`cadastrar_produto(estoque)`

opção 1 · R4 R5 R6

`exibir_total(estoque)`

opção 2 · R7

`buscar_produto(estoque, codigo)`

for + return · R5

`calcular_total_produtos(estoque)`

sum(map(lambda)) · R7

`ler_inteiro_nao_negativo · ler_preco_nao_negativo · ler_texto`
`try/except/raise · R6`

Ordem de construção

1. Dado (produto, estoque)
2. buscar_produto
3. ler_* validados
4. cadastrar_produto
5. calcular_total + exibir_total
6. exibir_menu + main
7. Roteiro de testes

De baixo para cima: cada função só usa o que já existe e já foi testado.

Funções pequenas com nome que diz o que fazem: é o que o enunciado chama de "organização e modularização".

1

A estrutura de dados

Um produto é um dicionário. O estoque é uma lista deles. (R3)

```
# teste no terminal interativo, antes das funções
produto = {
    "codigo": 101,
    "nome": "Teclado sem fio",
    "preco": 79.9,
    "quantidade": 10,
}
estoque = []
estoque.append(produto)
print(len(estoque))
1
print(estoque[0]["nome"])
Teclado sem fio
print(estoque[0]["preco"] * estoque[0]["quantidade"])
799.0
```

Por que dicionário?

produto["preco"] se lê sozinho.
produto[2] não. O avaliador precisa entender sem você do lado.

Erro que vai aparecer

```
estoque[0]["preço"]
KeyError: 'preço'
```

Chave é exata: sem acento, sem maiúscula. Padronize as 4 chaves agora.

buscar_produto(estoque, codigo)

Percorre a lista; devolve o dicionário ou None. É a base do "não duplicar" (R5)

```
def buscar_produto(estoque, codigo):  
    """Devolve o dicionário do produto, ou None se não existir."""  
    for produto in estoque:  
        if produto["codigo"] == codigo:  
            return produto          # achou: sai na hora  
    return None                    # percorreu tudo: não achou  
  
# teste  
estoque = [{"codigo": 101, "nome": "Teclado",  
              "preco": 79.9, "quantidade": 10}]  
print(buscar_produto(estoque, 101))  
{'codigo': 101, 'nome': 'Teclado', 'preco': 79.9,  
  'quantidade': 10}  
print(buscar_produto(estoque, 999))  
None
```

Por que None?

"Não achei" precisa ser um valor diferente de qualquer produto. None é isso.

Erro clássico

return None dentro do for (no else do if). Devolve None no 1º produto diferente — nunca chega no 2º.

Leitura validada: ler_inteiro / ler_preco

Prática 2 + prática 3 da aula 03, agora com nome definitivo (R6)

```
def ler_inteiro_nao_negativo(mensagem):
    while True:
        try:
            valor = int(input(mensagem))
            if valor < 0:
                raise ValueError("o valor não pode ser negativo")
            return valor
        except ValueError as e:
            print(f"  [!] Entrada inválida: {e}")

def ler_preco_nao_negativo(mensagem):
    while True:
        try:
            valor = float(input(mensagem).replace(",", "."))
            if valor < 0:
                raise ValueError("o preço não pode ser negativo")
            return valor
        except ValueError as e:
            print(f"  [!] Entrada inválida: {e}")
```

E o nome? ler_texto()

Mesma ideia, sem try: input().strip() e recusa vazio. Não há conversão que possa explodir. Código no roteiro, passo 3c.

Teste cada uma

```
ler_inteiro:
  abc → -5 → 2.5 → 10 ✓
ler_preco:
  xyz → -1 → 79,90 → 79.9 ✓
ler_texto:
  Enter → "  Mouse " → Mouse ✓
```

cadastrar_produto(estoque)

Junta buscar + ler_* + append. É a opção 1 do menu (R4, R5, R6)

```
def cadastrar_produto(estoque):  
    print("\n--- Cadastrar Produto ---")  
    codigo = ler_inteiro_nao_negativo("Código: ")  
  
    if buscar_produto(estoque, codigo) is not None:          # R5  
        print(f"  [!] Já existe um produto com o código {codigo}.")  
        return                                              # cancela  
  
    nome = ler_texto("Nome: ")  
    preco = ler_preco_nao_negativo("Preço (R$): ")          # R6  
    quantidade = ler_inteiro_nao_negativo("Quantidade: ")  # R6  
  
    produto = {"codigo": codigo, "nome": nome,  
               "preco": preco, "quantidade": quantidade}  
    estoque.append(produto)  
    print(f"  [ok] Produto '{nome}' cadastrado com sucesso.")
```

Por que checar o código primeiro?

Se já existe, não faz sentido perguntar nome e preço. return vazio encerra a função sem cadastrar.

Repare

A função não valida nada sozinha — delega para ler_*. E não mexe em variável global: estoque chegou como parâmetro.

calcular_total_produtos + exibir_total

Uma calcula e devolve; a outra mostra. É a opção 2 do menu (R7)

```
def calcular_total_produtos(estoque):  
    """Soma a quantidade de todos os produtos."""  
    return sum(map(lambda produto: produto["quantidade"], estoque))  
  
def exibir_total(estoque):  
    print("\n--- Total de Produtos em Estoque ---")  
    if len(estoque) == 0:  
        print(" Nenhum produto cadastrado ainda.")  
        return  
    total = calcular_total_produtos(estoque)  
    print(f" Produtos cadastrados: {len(estoque)}")  
    print(f" Total de unidades em estoque: {total}")
```

```
# equivalente sem lambda (também vale):  
total = 0  
for produto in estoque:  
    total += produto["quantidade"]  
return total
```

Por que 2 funções?

calcular devolve um número (dá para testar com print). exibir cuida da tela. Separar cálculo de exibição é o que o enunciado chama de modularização.

Teste

2 produtos, qtd 10 e 3
→ 13
lista vazia → 0
(sum de vazio é 0, não erro)

exibir_menu + main: o laço principal

while True + input + if/elif + break. Sem variável global (R1, R8)

```
def exibir_menu():
    print("\n==== CONTROLE DE ESTOQUE =====")
    print("1 - Cadastrar Produto")
    print("2 - Calcular Total de Produtos em Estoque")
    print("0 - Sair")

def main():
    estoque = []          # nasce aqui, morre quando o programa fecha
    (R8)
    while True:
        exibir_menu()
        opcao = input("Escolha uma opção: ").strip()
        if opcao == "1":
            cadastrar_produto(estoque)
        elif opcao == "2":
            exibir_total(estoque)
        elif opcao == "0":
            print("Encerrando o sistema.")
            break
        else:
            print("  [!] Opção inválida.")

main()                    # última linha do arquivo
```

opcao é texto

input() devolve str. Comparar com "1" (string) evita int() e o try. Simples e sem risco.

main() no fim!

Se chamar main() antes de definir as funções: NameError. Python lê de cima para baixo.

7

Roteiro de testes: prove que funciona

Rode esta sequência exata. Se a saída bater, os 8 requisitos estão atendidos

Passo	Digite	Saída esperada	Requisito
1	2	Nenhum produto cadastrado ainda.	R7 (vazio)
2	1 → Código: abc	[!] Entrada inválida: invalid literal for int()...	R6
3	Código: -5	[!] Entrada inválida: o valor não pode ser negativo	R6
4	101 · Teclado · 79,90 · 10	[ok] Produto 'Teclado' cadastrado com sucesso.	R3 R4
5	1 → Código: 101	[!] Já existe um produto com o código 101.	R5
6	1 → 102 · Mouse · -1	[!] Entrada inválida: o preço não pode ser negativo	R6
7	2,5 · 0	[ok] Produto 'Mouse' cadastrado com sucesso.	qtd 0 é válida
8	2	Produtos cadastrados: 2 / Total de unidades: 10	R7
9	9	[!] Opção inválida.	robustez
10	0	Encerrando o sistema.	R1

Oficina concluída

7 funções.

8 requisitos.

1 programa que não morre.

Para casa: rode o roteiro de testes mais uma vez, monte o PDF, entregue. Quem quiser ir além: opção 3 "Listar produtos" com `sorted(key=lambda p: p["preco"])`.

Enunciado: MIGNON, A. S. Produção Textual Individual — Práticas de Programação. Senac EAD. · Livro: Práticas de Programação, cap. 1–6, Editora Senac SP.